

Toc Definitions Unit1-3.5

Definition of ϵ -NFA (Epsilon-Nondeterministic Finite Automaton)

An ϵ -NFA (Epsilon-Nondeterministic Finite Automaton) is a type of **Nondeterministic Finite Automaton (NFA)** that allows ϵ (**epsilon**) **transitions**, meaning it can move to a new state without consuming any input symbol.

An ϵ -NFA is defined as a **5-tuple** $(Q, \Sigma, \delta, q_0, F)$, where:

- $Q \rightarrow$ Finite set of states
 - $\Sigma \rightarrow$ Finite input alphabet
 - $\delta \rightarrow$ **Transition function**: $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$ (maps a state and an input symbol or ϵ to a set of states)
 - $q_0 \rightarrow$ Initial state ($q_0 \in Q$)
 - $F \rightarrow$ Set of final states ($F \subseteq Q$)
-

Extended Transition Function of ϵ -NFA

The **extended transition function** δ^* for an ϵ -NFA extends δ to handle **strings**, not just single symbols. It is defined as:

1. **Base Case**: For any state q , $\delta^*(q, \epsilon)$ is the **ϵ -closure** of q , which includes all states reachable from q using only ϵ -moves.
2. **Recursive Case**: For a string $w = xa$, where x is a string and a is a symbol in Σ ,

$$\delta^*(q, xa) = \epsilon\text{-closure}\left(\bigcup_{p \in \delta^*(q, x)} \delta(p, a)\right)$$

This means we first compute all reachable states for x , then take transitions on a , and finally compute the **ϵ -closure** of the result.

Language Accepted by an ϵ -NFA

An **ϵ -NFA** accepts a string w if there exists a sequence of transitions (including possible ϵ -moves) that lead from the initial state q_0 to any final state in F .

Mathematically, the language L accepted by ϵ -NFA is:

$$L = \{w \in \Sigma^* \mid \delta^*(q_0, w) \cap F \neq \emptyset\}$$

This means that at least one state in $\delta^*(q_0, w)$ is a final state.

Key Points to Remember

- ϵ -transitions allow movement without consuming input.
- ϵ -closure is crucial in computing the extended transition function.
- ϵ -NFA can be converted to an equivalent NFA or DFA using the subset construction method.

a) Alphabets (Σ)

An **alphabet** is a finite, non-empty set of symbols used to form strings.

- It is denoted by Σ (Greek letter sigma).
 - Example: $\Sigma = \{0, 1\}$ (binary alphabet), $\Sigma = \{a, b, c, d\}$ (English letters).
-

b) Strings (w)

A **string** is a finite sequence of symbols chosen from an alphabet.

- A string can be **empty** (ϵ) or **non-empty**.
 - Example: If $\Sigma = \{a, b\}$, then "abba" and "baba" are strings.
 - **Length of a string**: Number of symbols in the string (denoted as $|w|$). Example: $|"abc"| = 3$.
-

c) Languages (L)

A **language** is a **set of strings** formed using an alphabet.

- A language L over an alphabet Σ is a subset of Σ^* (set of all possible strings).
 - **Types of Languages:**
 - **Finite Language:** Contains a limited number of strings.
 - Example: $L = \{ab, ba, bb\}$
 - **Infinite Language:** Contains an unlimited number of strings.
 - Example: $L = \{a^n \mid n \geq 0\}$ (all strings of only 'a').
 - **Example:** If $\Sigma = \{0,1\}$, a language L could be the set of all binary strings with an even number of 1s.
-

d) Grammar (G)

A **grammar** defines the **rules for generating strings** in a language.

- A grammar G is a 4-tuple (V, Σ, P, S) :
 - $V \rightarrow$ Set of variables (non-terminals).
 - $\Sigma \rightarrow$ Set of terminal symbols (alphabet).
 - $P \rightarrow$ Set of **production rules** (rewriting rules).
 - $S \rightarrow$ **Start symbol** (one of the variables).
- **Example Grammar (for balanced parentheses):**
 - $V = \{S\}, \Sigma = \{ (,) \}$
 - $P = \{ S \rightarrow (S) \mid SS \mid \epsilon \}$
 - S is the start symbol

This grammar generates strings like: "()", "(())", "()()", etc.

Definition of DFA (Deterministic Finite Automaton)

A **Deterministic Finite Automaton (DFA)** is a **finite state machine** that **accepts or rejects strings** by processing each input symbol exactly **one way**, without any ambiguity.

A DFA is defined as a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:

- $Q \rightarrow$ Finite set of states.
- $\Sigma \rightarrow$ Finite input alphabet.
- $\delta \rightarrow$ **Transition function**: $\delta : Q \times \Sigma \rightarrow Q$ (maps a state and an input symbol to exactly one state).
- $q_0 \rightarrow$ Initial state ($q_0 \in Q$).
- $F \rightarrow$ Set of final (accepting) states ($F \subseteq Q$).

A string is **accepted** by a DFA if, after processing all input symbols, the automaton ends in one of the **final states (F)**.

Different Notations for Describing DFA

1 Transition Table Notation

Represents the transition function δ in table form. Example:

State	Input: 0	Input: 1
$\rightarrow q_0$	q_1	q_2
q_1	q_0	q_3
$*q_2$	q_3	q_0
$*q_3$	q_2	q_1

Here,

- $\rightarrow q_0$ is the **initial state**.
- $*q_2, *q_3$ are **final states**.
- The table shows state transitions for inputs **0** and **1**.

2 Transition Diagram Notation

- **States** are represented as circles.
- **Arrows** show transitions for input symbols.
- **Final states** are marked with a **double circle**.

- **Initial state** has an **incoming arrow** from nowhere.

Example:

If $\Sigma = \{0,1\}$ and DFA accepts strings ending in **1**, the transition diagram would look like:

lua

```

→(q0) --0--> (q0)
    |
    1
    ↓
    (q1) --0,1--> (q1)

```

Here, q_0 is the start state, and q_1 is the final state.

3 Mathematical Notation

DFA can be written using set notation:

$$M = (Q, \Sigma, \delta, q_0, F)$$

where:

- $Q = \{q_0, q_1\}$
- $\Sigma = \{0,1\}$
- $\delta = \{ (q_0,0) \rightarrow q_0, (q_0,1) \rightarrow q_1, (q_1,0) \rightarrow q_1, (q_1,1) \rightarrow q_1 \}$
- q_0 is the start state
- $F = \{q_1\}$ (final state)

Key Points

- DFA has only one possible transition per state for each input symbol.
- DFA cannot have ϵ (epsilon) transitions.
- A string is accepted if it ends in a final state after processing all input symbols.

Definition of NFA (Nondeterministic Finite Automaton)

A **Nondeterministic Finite Automaton (NFA)** is a finite state machine where:

- From a given state, there can be **multiple transitions** for the same input symbol.
- It may also have **no valid transition** for some inputs.
- **NFA is defined as a 5-tuple** $(Q, \Sigma, \delta, q_0, F)$, where:
 - $Q \rightarrow$ Finite set of states.
 - $\Sigma \rightarrow$ Finite input alphabet.
 - $\delta \rightarrow$ **Transition function**: $\delta : Q \times \Sigma \rightarrow 2^Q$ (maps a state and an input to a **set of states**).
 - $q_0 \rightarrow$ Initial state ($q_0 \in Q$).
 - $F \rightarrow$ Set of final states ($F \subseteq Q$).

A string is **accepted** if at least one transition path reaches a final state.

Definition of ϵ -NFA (Epsilon-Nondeterministic Finite Automaton)

A **ϵ -NFA** is an NFA with **epsilon (ϵ) transitions**, meaning:

- The automaton can move to another state **without consuming any input symbol**.
- The transition function includes **ϵ -moves**:

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$$

- The **ϵ -closure** of a state includes **all states reachable through ϵ -transitions**.
-

Differences Between NFA and DFA

Feature	DFA (Deterministic Finite Automaton)	NFA (Nondeterministic Finite Automaton)
Transitions	Exactly one transition per input per state	Can have multiple transitions for the same input

Feature	DFA (Deterministic Finite Automaton)	NFA (Nondeterministic Finite Automaton)
Epsilon (ε) Transitions	✗ Not allowed	✓ Allowed in ε-NFA
State Change	Only one unique next state	Can move to multiple states at once
Complexity	More complex to design	Easier to construct
Conversion	Cannot be converted to NFA	Can be converted to DFA using subset construction
Processing	Always follows a single path	Explores multiple paths at once

Definition of Extended Transition Function (δ^*)*

The extended transition function δ^* allows an NFA or DFA to process entire strings instead of just single symbols. It is defined as:

For DFA:

- **Base Case:**

$$\delta^*(q, \varepsilon) = q$$

- **Recursive Case:**

$$\delta^*(q, xa) = \delta(\delta^*(q, x), a)$$

This means:

1. Compute $\delta^*(q, x)$ (where x is a prefix of the string).
2. Apply δ on the last symbol a .

For NFA/ε-NFA:

- **Base Case:**

$$\delta^*(q, \varepsilon) = \epsilon\text{-closure}(q)$$

- **Recursive Case:**

$$\delta^*(q, xa) = \epsilon\text{-closure}\left(\bigcup_{p \in \delta^*(q, x)} \delta(p, a)\right)$$

This means:

1. Compute **ϵ -closure** of the states reachable for x .
 2. Apply transitions for a .
 3. Compute **ϵ -closure** again for the new states.
-

Key Takeaways

- NFA allows multiple transitions per input, unlike DFA.
- ϵ -NFA allows transitions without consuming input.
- Extended transition function helps process strings step-by-step.
- Every NFA can be converted to an equivalent DFA.

Statement to Prove

For any **regular expression (RE)** r , there exists a **finite automaton (FA)** that accepts the **language $L(r)$** . This proves that **regular expressions and finite automata** are equivalent in expressive power.

Proof (Construction of Finite Automaton from RE)

A **Regular Expression (RE)** defines a **regular language**, and every regular language can be recognized by a **Finite Automaton (FA)**. The proof is by **constructing an NFA** for each type of regular expression.

Step 1: Base Cases

1 For ϵ (empty string):

- The NFA consists of a **single start and final state** with no transitions.

2 For a single symbol ($a \in \Sigma$):

- The NFA has two states:
 - q_0 (start state)

- q_1 (final state)
- A transition $q_0 \rightarrow q_1$ on a .

Step 2: Inductive Cases (Using Regular Operators)

Regular expressions are built using three operations: **Union** ($\cup$), **Concatenation**, and **Kleene Star** ($*$).

3 Union ($r_1 \cup r_2$) $\rightarrow$ Combine NFAs for r_1 and r_2

- Create a **new start state** q_{new} with **ϵ -transitions** to the start states of $\text{NFA}(r_1)$ and $\text{NFA}(r_2)$.
- The final states of $\text{NFA}(r_1)$ and $\text{NFA}(r_2)$ are connected to a **new final state** via **ϵ -transitions**.

4 Concatenation ($r_1 r_2$) $\rightarrow$ Connect NFAs sequentially

- Connect the **final states of $\text{NFA}(r_1)$** to the **start state of $\text{NFA}(r_2)$** using **ϵ -transitions**.

5 Kleene Star (r^*) $\rightarrow$ Repeat transitions using ϵ

- Create a **new start and final state**.
- Add **ϵ -transitions** from:
 - New start state $\rightarrow$ start state of $\text{NFA}(r)$.
 - Final states of $\text{NFA}(r) \rightarrow$ new final state.
 - Final states of $\text{NFA}(r) \rightarrow$ start state of $\text{NFA}(r)$ (loop).
 - New start state $\rightarrow$ new final state (to accept ϵ).

Conclusion

Since every RE can be converted into an NFA, and every NFA can be converted into a DFA, we conclude that:

For every regular expression r , there exists a finite automaton M that accepts $L(r)$.

Thus, regular expressions and finite automata are equivalent in power. 

Definition of Regular Language

A **regular language** is a language that can be **recognized by a finite automaton** (DFA or NFA). Formally, a language L over an alphabet Σ is **regular** if and only if there exists a **deterministic finite automaton (DFA)**, **nondeterministic finite automaton (NFA)**, or a **regular expression (RE)** that accepts it.

Proof: If L is a Regular Language, then there exists a Regular Expression r such that $L = L(r)$

We need to show that for every **regular language** L , there exists a **regular expression (RE)** r such that $L = L(r)$.

Step 1: Finite Automaton Representation

Since L is regular, by definition, there exists a **finite automaton (FA)** (either DFA or NFA) that recognizes L .

Step 2: Convert FA to Regular Expression

We use the **state elimination method** to construct a **regular expression** from a **finite automaton**:

- 1** Start with the given DFA or NFA representing L .
- 2** Introduce a new start state q_{start} (if needed) with **ϵ -transitions** to the original start state.
- 3** Introduce a new final state q_{final} with **ϵ -transitions** from the original final states.
- 4** Eliminate intermediate states one by one, replacing transitions with **regular expressions** using the formula:

$$R = R_1 R_2^* R_3$$

where R_1, R_2, R_3 are regular expressions representing possible paths.

- 5** After eliminating all intermediate states, we get a **single regular expression** representing L .
-

Conclusion

Since we have converted the **finite automaton (which accepts L)** into a **regular expression**, this proves that:

L is a regular language $\Rightarrow \exists$ a regular expression r such that $L = L(r)$.

Thus, regular languages and regular expressions are equivalent in expressive power .

Applications of Regular Expressions (REs)

Regular Expressions (REs) are widely used in various fields due to their ability to **match, search, and manipulate text efficiently**. Some key applications include:

1 Text Processing & Search

- Used in **text editors** (e.g., **VS Code, Sublime, Notepad++**) for **search and replace**.
- Helps in **finding patterns** in large documents (e.g., emails, phone numbers).

2 Compiler Design

- Lexical analyzers in compilers use **REs to tokenize source code** (identifying keywords, identifiers, numbers, etc.).
- Example: **Lex (Lexical Analyzer Generator)** uses REs to define token patterns.

3 Pattern Matching in Programming

- Used in **string processing functions** in programming languages like **Python, Java, JavaScript, Perl, etc.**
- Example: `re` module in Python allows pattern matching in text data.

4 Data Validation

- Validates **email addresses, phone numbers, IP addresses, and passwords** in web forms.
- Example: A regex for validating emails:

CSS

```
^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+$
```

5 Web Scraping & Data Extraction

- Extracts useful information (e.g., **URLs, HTML tags, product prices**) from websites.
- Used in web crawlers like **BeautifulSoup** and **Scrapy** in Python.

6 Network Security & Intrusion Detection

- Used in **firewalls and intrusion detection systems** (e.g., **Snort, Suricata**) to detect malicious patterns in network traffic.
- Helps in identifying spam, phishing URLs, and malware signatures.

7 Bioinformatics

- Used to analyze **DNA and protein sequences**, finding specific genetic patterns.
- Example: Identifying **ATG (start codon) sequences** in DNA sequences.

8 Log Analysis

- REs are used in **system log analysis** to detect error patterns, security breaches, or performance issues.

9 Search Engines

- Search engines like **Google, Elasticsearch** use REs for **efficient query processing** and **keyword extraction**.

Conclusion

Regular Expressions are powerful tools used in **text processing, security, data analysis, web scraping, programming, and more**. Their ability to efficiently **match, search, and extract patterns** makes them essential in various applications. ✓

Equivalence of States

Two states in a finite automaton (DFA or NFA) are said to be **equivalent** if they **recognize the same set of strings** (i.e., they lead to the same acceptance behavior for all input strings).

Formally, two states **p** and **q** are **equivalent** if, for every possible string **w**, both states lead to either:

- **Acceptance** (both reach a final state), or
- **Rejection** (both reach a non-final state).

Mathematically,

$$\delta^*(p, w) \text{ is a final state} \iff \delta^*(q, w) \text{ is a final state, for all } w \in \Sigma^*$$

Distinguishable States

Two states **p** and **q** are **distinguishable** if there exists at least **one string** w that, when processed from these states, leads to **different acceptance results**.

- If **one state reaches a final state** and the other **does not**, they are **distinguishable**.
- These states behave differently for some input and cannot be merged.

Indistinguishable States

Two states are **indistinguishable** if **no input string** w can differentiate them in terms of acceptance.

- If for **all strings** w , **both states behave identically** (either both accept or both reject), they are **indistinguishable**.
- Such states can be **merged** to minimize the DFA.

Example

Consider the following DFA:

State	Input: 0	Input: 1	Final?
q0	q1	q2	✗ No
q1	q3	q4	✓ Yes
q2	q3	q4	✓ Yes
q3	q3	q4	✓ Yes
q4	q4	q4	✗ No

- **q1 and q2 are indistinguishable**: Both always lead to acceptance.
- **q3 and q4 are distinguishable**: q3 leads to acceptance, while q4 does not.

By merging **indistinguishable states**, we can **minimize the DFA**. ✓

1 Context-Free Grammar (CFG)

A **Context-Free Grammar (CFG)** is a formal grammar that defines a language using **production rules**. It is represented as:

$$G = (V, \Sigma, P, S)$$

where:

- $V \rightarrow$ Set of **non-terminal symbols**.
- $\Sigma \rightarrow$ Set of **terminal symbols** (alphabet).
- $P \rightarrow$ Set of **production rules** of the form $A \rightarrow \alpha$, where A is a non-terminal, and α is a string of terminals & non-terminals.
- $S \rightarrow$ **Start symbol** (one of the non-terminals).

Example:

$$S \rightarrow aSb \mid \epsilon$$

This generates strings like ϵ , **ab**, **aabb**, **aaabbb**,

2 Leftmost Derivation

A **leftmost derivation** is a sequence of derivation steps where the **leftmost non-terminal** is **always replaced first**.

Example for $S \rightarrow aSb \mid \epsilon$:

1 $S \rightarrow aSb$

2 $aSb \rightarrow aaSbb$

3 $aaSbb \rightarrow aabb$

Here, the leftmost S is always replaced first.

3 Rightmost Derivation

A **rightmost derivation** replaces the **rightmost non-terminal first** at each step.

For the same grammar:

1 $S \rightarrow aSb$

2 $aSb \rightarrow ab$

3 ab

Here, the **rightmost** S is always replaced first.

4 Language of a Grammar

The language generated by a CFG is the set of all strings **derivable** from the start symbol using production rules.

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$$

For $S \rightarrow aSb \mid \epsilon$, the language consists of **balanced strings of 'a's and 'b's** like:

$$L = \{\epsilon, ab, aabb, aaabbb, \dots\}$$

5 Sentential Form

A **sentential form** is any intermediate string that appears in a **derivation** from the start symbol.

Example:

For $S \rightarrow aSb \mid \epsilon$, the **sentential forms** could be:

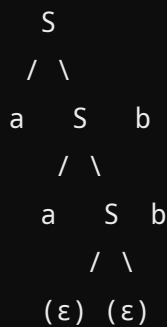
- S (Start)
 - aSb
 - $aaSbb$
 - $aabb$ (Final string)
-

6 Derivation Tree / Parse Tree

A **derivation tree** (or **parse tree**) visually represents how a string is derived using grammar rules.

Example for "aabb" with $S \rightarrow aSb \mid \epsilon$:

CSS



- **Root** = Start symbol S .
- **Leaves** = Terminals forming the string.
- **Branches** = Applications of production rules.

Both **derivation trees** and **parse trees** help visualize how a string is generated in a CFG! ✓

1 Chomsky Normal Form (CNF)

A CFG is in **Chomsky Normal Form (CNF)** if all its production rules are of the form:

1. $A \rightarrow BC$ (where A, B, C are non-terminals, and $B, C \neq S$)
2. $A \rightarrow a$ (where a is a terminal)
3. $S \rightarrow \epsilon$ (Only allowed if S does not appear on the right-hand side of any rule)

Example:

✓ CNF Grammar

$$S \rightarrow AB, \quad A \rightarrow a, \quad B \rightarrow b$$

Generates "ab".

2 Greibach Normal Form (GNF)

A CFG is in **Greibach Normal Form (GNF)** if all its production rules are of the form:

$$A \rightarrow a\alpha$$

where a is a terminal, and α is a sequence of non-terminals.

Example:

✓ GNF Grammar

$$S \rightarrow aS, \quad S \rightarrow b$$

Generates strings "aab", "ab", "b", etc.

3 Useless Productions

A **useless production** is a production rule that:

1. Never contributes to a final string (i.e., cannot derive a terminal string).
2. Cannot be reached from the start symbol.

Example:

✗ Useless Production

$$S \rightarrow AB, \quad A \rightarrow a, \quad B \rightarrow C, \quad C \rightarrow D$$

Here, D has no production rule, making B and C useless.

4 Unit Productions

A **unit production** is a rule of the form:

$$A \rightarrow B$$

where both A and B are non-terminals.

Example:

✗ Unit Production

$$S \rightarrow A, \quad A \rightarrow B, \quad B \rightarrow a$$

Can be replaced directly as:

✓ Simplified

$$S \rightarrow a$$

5 λ -Productions (ϵ -Productions)

A λ -production (or ϵ -production) is a rule that allows a non-terminal to **disappear** (derive an empty string).

Example:

$$A \rightarrow \epsilon$$

Removing λ -productions helps simplify a grammar and convert it into CNF. ✓

1 Pushdown Automaton (PDA)

A Pushdown Automaton (PDA) is a finite state machine with a **stack** that allows it to recognize **context-free languages (CFLs)**.

A PDA is defined as:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

where:

- $Q \rightarrow$ Set of states
- $\Sigma \rightarrow$ Input alphabet
- $\Gamma \rightarrow$ Stack alphabet
- $\delta \rightarrow$ Transition function: $\delta(q, a, X) \rightarrow (q', \gamma)$
- $q_0 \rightarrow$ Initial state
- $Z_0 \rightarrow$ Initial stack symbol
- $F \rightarrow$ Set of final states

✓ **Example:** PDA for $L = \{a^n b^n \mid n \geq 0\}$ uses a **stack** to track the number of 'a's and ensures it matches the number of 'b's.

2 Nondeterministic Pushdown Automaton (NPDA)

An NPDA is a PDA that allows multiple possible transitions from the same state for a given input.

- Unlike a deterministic PDA (DPDA), an NPDA can guess the correct path to acceptance.
- NPDA recognizes all context-free languages, whereas DPDA recognizes only a subset of them.

✓ **Example:** NPDA for $L = \{ww^R \mid w \in \{a, b\}^*\}$ (Palindrome language).

3 Turing Machine (TM)

A Turing Machine (TM) is a mathematical model of computation that can simulate any algorithm. It consists of:

- Infinite tape (memory).
- Read/write head (moves left or right).
- Finite control (states and transitions).

A TM is defined as:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$$

where:

- $Q \rightarrow$ Set of states
- $\Sigma \rightarrow$ Input alphabet
- $\Gamma \rightarrow$ Tape alphabet (includes $\square$ for blank)
- $\delta \rightarrow$ Transition function $\delta(q, a) = (q', b, D)$, where:
 - $q' \rightarrow$ New state
 - $b \rightarrow$ Symbol to write
 - $D \rightarrow$ Move Left (L) or Right (R)
- $q_0 \rightarrow$ Start state
- $q_{accept} \rightarrow$ Accept state

- $q_{reject} \rightarrow$ Reject state

✓ **Example:** A TM that checks if a binary string has an equal number of 0s and 1s.

4 Counter Machine

A Counter Machine is a simplified Turing Machine with only a finite-state control and one or more counters instead of an infinite tape.

- Each counter stores a non-negative integer.
- Operations allowed: Increment, Decrement, and Zero-Check.
- Can simulate a PDA, but is less powerful than a TM.

✓ **Example:** A 2-counter machine can recognize $L = \{a^n b^n\}$ by using one counter to count 'a's and decrementing it when reading 'b's. ✓